# **D**S3040 Introduction to Deep Learning

Assignment-9, Date: Apr 6, 2026

Timing: 9:00 to 11:45 AM | Max Marks: 15

---

**Instructions**

1. Submit one .ipynb file containing all answers. The name should be [student_name] _assignment[number].ipynb
2. Write the questions in separate text blocks before the answers.
3. Outputs for all sub-questions should be given and the code should be executable.
4. Write justifications for your choices where needed.
5. Ensure that all plots include clear labels and legends for better interpretation.
6. Do not use AI or the internet. Refer only library documentations.

---

In this assignment you will implement Flow Matching from scratch to generate points from synthetic distributions and MNIST images.

1. Train a Flow Matching model to generate synthetic data (**any one of half moon, swiss roll or circles**). (6)

   a Implement sinusoidal position encoding for the time input, and a neural network which takes a data point $x$ and time $t$ as inputs to calculate the vector field at that time.

   You should project $x$ using a linear layer to a hidden dimension, and embed $t$ using positional encoding followed by a projection to the hidden dimension. The encoded inputs can then be passed through linear layers with ReLU activation followed by a final projection to the required output dimension. (2)

   b Train the Flow Matching model with the above network to generate synthetic data. (2)

   c Generate 100 data points by simulating the ODE using Euler method with 1000 steps. Plot the intermediate generated points after every 100 steps to see the noise getting converted to data samples and the final generated data. (2)

2. Train a Flow Matching model to generate MNIST images of **any one class**. (9)

   a Implement a UNet which takes an image and time as inputs to calculate the vector field at that time. The UNet can be 2 levels deep, using SiLU (swish) activation function. At each skip connection, pass a time embedding with the same size as the prepooled feature map in addition to the prepooled feature

map to the corresponding expanding path block. (Refer " Useful Information " for code snippets to implement the UNet. You have to modify the input sizes and forward pass of the upsampling block and UNET_Vector_Field to account for the time embedding. Define the model so that the bottleneck layer receives an input with 128 channels and outputs 256 channels.) (4)

b Train the Flow Matching model with the above network to generate images. Train for at least 3 epochs. (2)

c Generate an image from noise by simulating the ODE using Euler method with 1000 steps. Display the intermediate image after every 100 steps during generation for a single image to see the noise getting converted into a data sample. (3)

**Useful information**

- Generating synthetic data

```
import sklearn.datasets as ds
data, labels = ds.make_moons(n_samples=1000, shuffle=True, noise=0.03,
                             random_state=None)
data, labels = ds.make_circles(n_samples1000N,shuffle=True,noise=0.03,
                             random_state=None)
X, t = ds.make_swiss_roll(n_samples=1000, noise=0.03, random_state=None)
X = X[:, [0, 2]]
```

- Sinusoidal position encoding formula

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

- Sampling time point and initial noise

```
# sampling time from a uniform distribution
torch.rand()
# sampling noise from a normal distribution
torch.randn(size)
```

- Creating a subset of MNIST images of a particular class

```
mnist = datasets.mnist.MNIST(root = "", download = False, train = True, transform=
transforms.Compose([transforms.ToTensor(),transforms.Normalize(0.5, 0.5)]))

test_idx = [idx for idx, target in enumerate(mnist.targets) if target == t]

subset = Subset(mnist, indices = test_idx)
train_dataloader = DataLoader(subset, batch_size = 64)
```

- Algorithms for Flow Matching training and data sampling

**Algorithm 3** Flow Matching Training Procedure (for Gaussian CondOT path $p_t(x|z) = \mathcal{N}(tz, (1-t)^2)$)

**Require:** A dataset of samples $z \sim p_{\text{data}}$, neural network $u_t^\theta$

**for** each mini-batch of data **do**
  Sample a data example $z$ from the dataset.
  Sample a random time $t \sim \text{Unif}_{[0,1]}$.
  Sample noise $\epsilon \sim \mathcal{N}(0, I_d)$
  Set

$$x = tz + (1-t)\epsilon \qquad \text{(General case: } x \sim p_t(\cdot \mid z))$$

  Compute loss

$$\mathcal{L}(\theta) = \|u_t^\theta(x) - (z - \epsilon)\|^2 \qquad \text{(General case: } = \|u_t^\theta(x) - u_t^{\text{target}}(x|z)\|^2)$$

  Update $\theta \leftarrow \text{grad_update}(\mathcal{L}(\theta))$.
**end for**

**Algorithm 1** Sampling from a Flow Model with Euler method

**Require:** Neural network vector field $u_t^\theta$, number of steps $n$

Set $t = 0$
Set step size $h = \frac{1}{n}$
Draw a sample $X_0 \sim p_{\text{init}}$
**for** $i = 1, \ldots, n$ **do**
  $X_{t+h} = X_t + h u_t^\theta(X_t)$
  Update $t \leftarrow t + h$
**end for**
**return** $X_1$

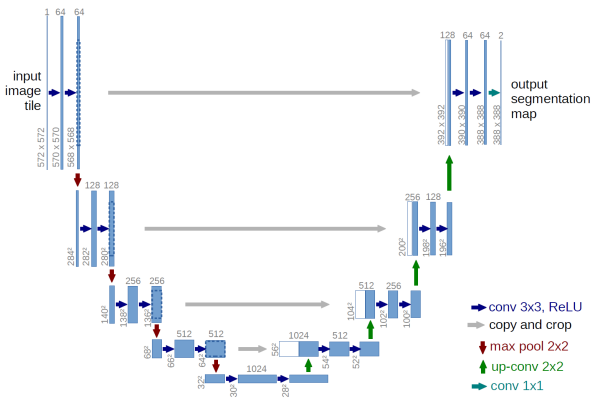
- You will be modifying the UNet architecture to add time encodings in the skip connections to learn the velocity field for image generation.

  The U-Net architecture consists of two main components: a contracting path (encoder) and an expanding path (decoder), which together form a symmetric, U-shaped structure.

  The contracting path is composed of downsampling blocks, which progressively reduce the spatial dimensions while increasing the number of feature channels.

  At the bottom of the U, there's a bottleneck layer that captures the most abstract representation of the input.

  The expanding path consists of upsampling blocks, which gradually recover the spatial dimensions and refine the output.



- Starter code for the UNet model:

```
class convolution_block(nn.Module):
  def __init__(self, input_channels, output_channels):
    super().__init__()
    #define the operations inside the block
    self.block = nn.Sequential(
    nn.Conv2d(input_channels,output_channels,kernel_size=3,padding=1,bias=False),
    nn.BatchNorm2d(output_channels),
    nn.Conv2d(output_channels,output_channels,kernel_size=3,padding=1,bias=False),
    nn.BatchNorm2d(output_channels),
    nn.SiLU(inplace=True)
    )

  def forward(self,x):
      x = self.block(x)
      return x

---------------------------------------------------------------------------

class downsampling_block(nn.Module):
  def __init__(self, input_channels, output_channels):
    """
    Args:
        input_channels (int): The number of channels in the input feature map.
        output_channels (int): The number of channels after the convolution block.
    """
    super().__init__()
    self.conv_block = convolution_block(input_channels, output_channels)
    self.pool = nn.MaxPool2d((2,2))

  def forward(self,x):
    x = self.conv_block(x)
    p = self.pool(x)
    # Return both the pre-pooled feature map for the skip connection
    # and the pooled feature map for the next layer in the encoder.
    return x,p

---------------------------------------------------------------------------


class upsampling_block(nn.Module):
  """
    An upsampling block for the U-Net expansive path.
```

```
    It combines upsampling with a skip connection from the encoder path.
  """
  def __init__(self, input_channels, output_channels):
    super().__init__()
    #define the up-convolution
    self.up_convolution = nn.ConvTranspose2d(input_channels, output_channels,
                                    kernel_size=2, stride=2, padding=0)
    #define the convolution: current output + skip connection
    self.convolution_block = convolution_block(
                           output_channels+output_channels, #current + skip
                           output_channels
                           )
  def forward(self,x,skip_connection):
    x = self.up_convolution(x)
    #concatenate the previous result with the skip connection before applying
    # the convolution block
    x = torch.cat([x,skip_connection],axis=1)
    #apply the convolution
    x = self.convolution_block(x)
    return x

---------------------------------------------------------------------------

class UNET_Vector_Field(nn.Module):
  def __init__(self):
    super().__init__()
    '''
      TIME ENCODINGS FOR THE SKIP CONNECTION
    '''
    self.t1 =
    self.t2 =
    '''
      CONTRACTING PATH
    '''
    self.encoder_1 = downsampling_block(,)
    self.encoder_2 = downsampling_block(,)
    '''
      BOTTLENECK
    '''
    self.bottleneck = convolution_block(,)

    '''
      EXPANSIVE PATH
    '''
    self.decoder_1 = upsampling_block(,)
```

```
    self.decoder_2 = upsampling_block(,)

    '''
      OUTPUT LAYER
    '''
    self.output_layer = nn.Conv2d(,,kernel_size=1)

  def forward(self,x):
    #contracting path
    x1,p1 = self.encoder_1(x)
    x2,p2 = self.encoder_2(p1)

    #bottleneck
    b = self.bottleneck(p2)

    #expanding path
    # Add time encodings here
    d1 = self.decoder_1(b,x2) #use of skip connection
    d2 = self.decoder_2(d1,x1) #use of skip connection

    #output layer
    output = self.output_layer(d2)

    return output
```